LLARS - Legacy Script Creator Guide

Maintenance Reference for Traditional AutoHotkey v1 Code

Current for LLARS v2

Scope - This guide documents the public LLARS project as it operates in v2. Follow the shared framework and the current GUI/API behavior described here.

Contents

1. What This Guide Is For
2. What Must Stay Framework-Owned
3. Traditional RunCount Lifecycle
4. Traditional Config Reads
5. Traditional Randomization and Sleeps
6. Keyboard Migration
7. Mouse Migration
8. Pixel Reads and Detection
9. Traditional SetTimer Labels
10. Managed Label Timer Compatibility
11. Timed Script Lifecycle
12. Config Metadata Still Matters
13. RuneScape Focus Responsibilities
14. Developer Mode and Diagnostics
15. Pause, Exit, and Run-State Guards
16. Legacy Helper Reference
17. When Lower-Level Code Is Reasonable
18. When to Migrate
19. Maintenance Example
20. The Simple Version

1. What This Guide Is For

This guide helps maintain scripts that still contain traditional AutoHotkey v1 patterns such as labels, IniRead, Random, direct Sleep, or label SetTimer. It is not the default starting point for new public LLARS scripts.

Current contribution boundary - Raw game-facing physical input is not accepted in current public Scripts/Templates. When maintaining a legacy Send/Click/MouseMove path, migrate that physical action to LLARS_Press*, LLARS_Click*, or another shared LLARS helper.

2. What Must Stay Framework-Owned

- LLARS initialization and project discovery.
- Run lifecycle, pause/exit state, and shared timer cleanup.
- The selected RuneScape run window and game-facing input routing.
- Typed setup/config validation where a shared helper exists.
- Support diagnostics and structured warning/error/recovery reporting.

3. Traditional RunCount Lifecycle

Existing scripts may still use labels and first-run variables. Runtime estimation recognizes compatible patterns, but current code is easier to reason about when migrated to LLARS_RunCount(Func("Callback")).

```
; Preferred migration target:
Start:
```

```
LLARS_RunCount(Func("RunTask"))
return

RunTask(ctx)
{
    if (ctx.IsFirst)
        LLARS_LogEvent("Run started")
    ; activity logic
}
```

4. Traditional Config Reads

A maintained legacy script may contain IniRead. Before adding another direct read, check whether a typed LLARS_ConfigRead* helper covers the value.

```
; Traditional
IniRead, option, Config.ini, Feature, option, false

; Current
option := LLARS_ConfigReadBool("Feature")
```

5. Traditional Randomization and Sleeps

```
; Traditional maintained code may contain:
Random, waitMs, 800, 1400
Sleep, %waitMs%

; Current configurable timing:
LLARS_Sleep("Action Delay")
```

Current timing helpers preserve run-state behavior and make RunCount estimates more accurate. Prefer them when converting code.

6. Keyboard Migration

Traditional Send bypasses LLARS input routing, AHI/Standard fallback, focus safety, and transport diagnostics. Replace it in public Scripts/Templates.

```
; Legacy
; Send, {Space}

; Current
LLARS_PressKey("Space")
LLARS_PressHotkey("Action Hotkey")
```

7. Mouse Migration

Traditional MouseMove/Click has the same boundary problem as Send. Do not preserve raw physical mouse input in current public Scripts/Templates simply by placing LLARS_WaitForRuneScape() before it.

```
; Configured action
LLARS_Click("Action Location")
```

```
; Resolved target point
LLARS_ClickPoint(x, y, "left", "Detected target")

; Verified RuneScape click
LLARS_RedClick("Action Location")
```

8. Pixel Reads and Detection

Legacy PixelGetColor can be migrated to LLARS_PixelColor / LLARS_PixelMatches* for configured pixels. For advanced searches use shared LLARS_Color* capture/tracking helpers rather than adding new script-local Win32 capture code.

```
current := LLARS_PixelColor("Pixel Coordinate")
if LLARS_PixelMatches("Pixel Coordinate", "Target Color")
    LLARS_SetDetection("Target", "Matched")
```

9. Traditional SetTimer Labels

Existing scripts can still contain AutoHotkey label timers. They remain the right choice for framework-owned GUI countdown labels and may remain in maintained activity code where conversion adds no value. New activity callbacks should normally use managed timers.

10. Managed Label Timer Compatibility

LLARS_TimerSchedule(labelName, section, repeat, scope) exists for compatibility with label callbacks. For new code prefer LLARS_TimerOnce/LLARS_TimerRepeat with Func objects.

11. Timed Script Lifecycle

```
if (!LLARS_StartTimerRun())
    return

; ... run timer/callback work ...

LLARS_TimerStopAll()
LLARS_EndTimerRun()
```

12. Config Metadata Still Matters

A legacy implementation does not exempt the script from public configuration rules. Coordinate/hotkey/color metadata drives the setup GUI, status, reset behavior, and diagnostics. Keep dependencies explicit and valid.

13. RuneScape Focus Responsibilities

LLARS_WaitForRuneScape() is available for specialized non-input work that should occur only while the selected RuneScape client is foreground. It must not be used as a wrapper around raw Send/Click/MouseMove in current public scripts. Physical input still goes through shared input helpers.

14. Developer Mode and Diagnostics

Traditional code is less observable if it bypasses shared helpers. Migrate meaningful events to the current diagnostics API:

```
LLARS_LogEvent("State changed", "Optional detail")
LLARS_SetDetection("Target", "Searching")
LLARS_Warn("Script Warning", "What happened", "What LLARS did", "What the user should do",
"Optional detail", false)
LLARS_Recovered("Recovered target", "Nearby scan")
```

API name - Use LLARS_LogEvent(). A generic logger named Log is not part of the current Creator API.

15. Pause, Exit, and Run-State Guards

Callbacks that can fire outside the main loop should begin with LLARS_RunActive() when they require an active run. Managed timers are stopped by shared cleanup, but script-owned timers must still be turned off at the correct lifecycle boundary.

16. Legacy Helper Reference

Traditional pattern	Current LLARS direction
IniRead	LLARS_ConfigRead* when typed behavior exists
Random + Sleep	LLARS_Sleep / managed timer interval
Send	LLARS_PressKey / LLARS_PressHotkey
Click / MouseMove	LLARS_Click* / shared detection and target helpers
PixelGetColor	LLARS_PixelColor / LLARS_PixelMatches*
SetTimer label	LLARS_TimerRepeat/Once callback where appropriate
generic custom logger	LLARS_LogEvent
ad-hoc MsgBox error	LLARS_Warn / LLARS_Error

17. When Lower-Level Code Is Reasonable

- Pure computation or data transformation with no shared helper needed.
- AutoHotkey GUI behavior local to the script where it does not duplicate a shared LLARS control.
- Compatibility maintenance in an existing script while preserving the current public input/config/lifecycle boundaries.
- A non-input Windows operation with no shared abstraction, after confirming it does not create a direct game-input path.

18. When to Migrate

- The code sends physical keyboard or mouse input.
- The code duplicates focus/window validation.
- The code implements its own configured timing, timer cleanup, or support logging.
- The code reads typed setup values that have a shared reader.
- The code needs detection state visible in Developer Mode/support output.

19. Maintenance Example

```
LegacyAction()
{
    if !LLARS_RunActive()
        return

    ; Existing local calculations can stay local.
    mode := LLARS_ConfigReadChoice("Options", "mode", ["One", "Two"], "One")
    LLARS_SetStatus("Working", mode)
    LLARS_Click("Action Location")
    LLARS_Sleep("Action Delay")
    LLARS_LogEvent("Legacy action complete", mode)
}
```

20. The Simple Version

Traditional AutoHotkey syntax can remain where it is truly local implementation detail. For current public LLARS scripts, shared framework ownership still wins at the boundaries: configuration, lifecycle, RuneScape targeting, physical input, timers, and diagnostics.